

Guia Técnico — Diretrizes para Entrega do Desafio

Objetivo da Entrega

A proposta deste desafio é avaliar uma implementação com nível equivalente a Desenvolvedor Pleno (ou próximo ao nível Sênior). Espera-se que o candidato demonstre organização arquitetural, domínio básico de DDD, testes bem estruturados, código limpo, documentação clara e ambiente containerizado.

1. Estrutura Recomendada do Projeto

Evite concentrar toda a lógica em Controller e Service. Recomenda-se a seguinte organização:

```
controller/  
service/  
domain/  
repository/  
dto/  
exception/  
config/
```

A camada Domain deve concentrar regras de negócio relevantes, incluindo métodos de criação, exclusão e validações encapsuladas na entidade.

```
public class Coupon {  
  
    public static Coupon create(...) {  
        validateExpirationDate();  
        validateDiscountValue();  
        sanitizeCode();  
        return new Coupon(...);  
    }  
  
    public void delete() {  
        if(this.deleted) {  
            throw new BusinessException("Coupon already deleted");  
        }  
        this.deleted = true;  
    }  
}
```

2. Pontos Técnicos Avaliados

Sanitização do código: remover caracteres especiais e garantir 6 caracteres utilizando regex.

```
code.replaceAll("[^a-zA-Z0-9]", "")
```

Validação de expiração: cupons não devem possuir data passada.

```
if(expirationDate.isBefore(LocalDate.now())) {  
    throw new BusinessException("Expiration date cannot be in the past");  
}
```

Soft delete: utilizar campo booleano 'deleted', evitar remoção física do banco e garantir filtro automático nos repositórios.

```
@Where(clause = "deleted = false")
```

3. Testes

Busca-se cobertura próxima ou superior a 80%. Devem existir testes de domínio que validem criação, regras de desconto, sanitização e exclusão. Ferramentas sugeridas: JUnit 5 e Mockito.

4. Containerização (Docker)

```
FROM eclipse-temurin:17
WORKDIR /app
COPY target/app.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]

version: '3'
services:
  app:
    build: .
    ports:
      - "8080:8080"
```

5. Documentação da API (Swagger)

Adicionar a dependência springdoc-openapi-starter-webmvc-ui e disponibilizar em /swagger-ui.html.

6. Boas Práticas Esperadas

- Utilizar DTO separado da Entity;
- Criar classe BusinessException;
- Implementar GlobalExceptionHandler com @ControllerAdvice;
- Utilizar ResponseEntity corretamente;
- Padronizar respostas da API.

Diferenciais para Nível Próximo ao Sênior

- Bean Validation (@NotNull, @Size);
- Testes de integração com @SpringBootTest;
- README estruturado com instruções de execução, Docker, endpoints e decisões técnicas.

Pontos que Impactam Negativamente a Avaliação

- Regras de negócio no Controller;
- Soft delete incorreto;
- Ausência de testes de regra de negócio;
- Tratamento inadequado de exceções;
- Código procedural concentrado no Service.

Estratégia Recomendada

Durante o desenvolvimento, considere: "Se eu estivesse avaliando esta entrega, quais evidências demonstrariam maturidade técnica?" Essa abordagem contribui para uma entrega mais consistente e profissional.